

Janiya Green

CW-02 Critical Thinking Questions

Github: https://github.com/xxklusivenana/movie_watchlist_app/tree/master

1. Navigator Stack Mental Model

Flutter's Navigator works like a stack of cards. When you call `Navigator.push()`, a new screen is placed on top. When the user taps Back, it's `Navigator.pop()`. Draw or describe what the Navigator stack looks like in your app at three different moments: on `HomeScreen`, after navigating to `DetailsScreen`, and after navigating to `WatchlistScreen`. How does understanding this model help you manage navigation correctly?

- Home Screen : `Navigator.push()`
 - [`HomeScreen`] current screen
- Details Screen : `Navigator.push()`
 - [`HomeScreen`]
 - [`DetailsScreen`] current screen
- Transition Back : `Navigator.pop()`
 - [`HomeScreen`] current screen

2. StatelessWidget vs StatefulWidget

You used `StatelessWidget` for `DetailsScreen` initially, but when adding the watchlist toggle you had to change to `StatefulWidget`. Why? Explain the difference between these two widget types and describe a rule of thumb you would use to decide which one to use when designing any new screen in a Flutter app.

- I'm not a grad student, so I didn't need to create the watchlist toggle.
- Either way, the watchlist toggle would need to be stateful so it is seen as a dynamic widget. It changes the screen and updates the UI and these changes need to be saved with `setState()`. The real difference between stateful and stateless widgets is that stateless widgets are for context and setup (they don't change), while stateful widgets are dynamic and interactive.
- Rule of thumb: If something is only being displayed, use a stateless widget. If the screen needs to change, update, or remember something, use a stateful widget.

3. Data Architecture: Model vs Hard-coded

You created a `Movie` model class and a separate data file. Compare this approach to hard-coding movie text directly inside `DetailsScreen`. What are the advantages of a model-driven architecture? If you were building a production app that fetched movies from a REST API, what would need to change in your current architecture?

- The model-driven architecture I implemented makes the app easier to organize and update. I can add another movie to the data file without rewriting the `DetailsScreen`. It also allows the same movie information to be used by multiple screens.
- If I used a REST API in a production app, I would replace the static movie data with an API/service layer. The app would request movie data from the API, convert the JSON response into `Movie` objects, and then pass those objects to the screens.

4. Asset Management in Flutter

Describe the complete process of adding an image asset to a Flutter app from placing the file to displaying it on screen. Why must assets be declared in pubspec.yaml? What are two potential pitfalls a developer might encounter, and how would they debug them?

- I start by creating a folder under assets called images and download all my photos into there. Then, I go into pubspec.yaml and add each asset using

```
64   assets:  
65     - assets/images/      # includes all files in folder  
66     - assets/images/inception.jpg |  
67     - assets/images/buffy.jpg  
68     - assets/images/hollow_man.jpg  
69     - assets/images/jawbreaker.jpg  
70     - assets/images/jumanji.jpg  
71     - assets/images/mummy.jpg
```

To finally display them in the app, I do to the dart page I want them to be on (homepage_screen.dart) and

```
leading: ClipRect(  
  borderRadius: BorderRadius.circular(8),  
  child: Image.asset(  
    movie.posterPath,  
    width: 56,  
    height: 80,  
    fit: BoxFit.cover,  
  ),
```

add them using Once i'm ready for them to show in the app, I run flutter pub get in the terminal.

- Assets have to be displayed in pubspec.yaml so the app knows which files it's using. Two potential pitfalls a developer could encounter would be incorrectly adding file to yaml or typing an incorrect file path. A way to debug the path issue would be directly copy/pathing asset paths and making sure they are the same in all files used. For any issues with the yaml file, I'd recommend checking indentation, any base code comments, and the file path.

5. UX Design Decisions

You were given freedom in designing the UI ("This will be your design")

-. What design decisions did you make for your HomeScreen and DetailsScreen?

Consider: colour scheme, typography, layout, spacing, and use of posters. What design principles (e.g., visual hierarchy, consistency, affordance) guided your choices? What would you improve if you had more time?

- For my homescreen, I decided to use a simple design. Each movie is displayed CARD with a rounded poster on the left, the movie title is in bold and a chevron on the right. I used ListView.builder so the movies can be displayed in a scrollable list.

- For my Detailed screen, I centered the movie poster at the top and used larger bold text for the movie title. I put the cast and synopsis into different sections using spacing and larger headings. The poster also has rounded corners. My main design focused more on visuals because I wanted the movie poster and the title to stand out first, followed by the cast and synopsis, I also made sure to keep all spacing and typography consistent between sections.

6. State Persistence & Scalability

The watchlist in this app resets every time the app is restarted. For a real app, how would you make this data persist? Research and briefly describe two approaches: (1) using Flutter's `shared_preferences` package, and (2) using a local database like `sqflite`. What are the trade-offs between these two approaches?

- I would use `shared_preference` this can store small amounts of data locally, such as list of movie id or title that the user saved and the pros include easy to set up, no complicated database needed while some cons are these are not designed for large or complicated datasets and can also be more difficult to manage the relationships between different types of data.
- `Sqflite` provides a local `SQLite` database. I could create a movie table and a watchlist table and store the movie information in the database. The advantages include better for larger amounts of data, supports queries and structured data, while the disadvantages include more advanced setup and code needed and it requires managing database tables, queries and updates.

7. Git Workflow & Commit Culture

This assignment required continuous development via GitHub, not a bulk upload. Why do you think the instructor requires this? Reflect on your own commit history: were your commits small and frequent, or large and occasional? Write two of your actual commit messages and evaluate whether they follow best practices. How would a strong commit history benefit a future employer reviewing your portfolio?

- I think it's required to get us used to writing commit messages. My commits were more occasional— after I finished stages of the app.
- movie watchlist app base code and added main.dart, tested, added app screenshots
- I think they follow best practices. A strong commit history can help a future employer see that I understand version control, can work incrementally, and can organize development into meaningful changes. It also makes it easier to find when a bug or change was introduced.

8. Beyond the Assignment: Feature Brainstorm

Imagine you are pitching this Movie Watchlist app to a product manager. Propose three new features you would add (beyond the assignment requirements). For each feature, describe: (1) what it does from the user's perspective, (2) which Flutter widgets or packages you would use to implement it, and (3) why it adds value to the user experience.

- Feature 1: Saved

- From the user's perspective it helps to look back at past movies they'd liked, watched, and might want to return to in the future.
- I'd implement it using
- It adds value to the user experience by
- Feature 2: Search Bar
 - From the user's perspective it's help to be able to quickly look for movies instead of just having to continuously scroll a long list.
 - I'd implement it using a text field with ListView.builder and filter based on the search text.
 - It adds value to the user experience by improving user experience.
- Feature 3: Ratings/Reviews
 - From the user's perspective it shows real, personal opinions and feedback. It keeps track of how they felt/thought about a movie and they can look back on it later. There could even come a time where individual reviews are rating are shared with other users.
 - I'd implement it using a RatingBar, icons, and a text field. The data would be saved and sent to the system using something like sqflite or shared_preferences.
 - It adds value to the user experience by